



DATA SATURDAY 2026

CENTRO UNIVERSITÁRIO SALESIANO
AV. VITÓRIA, 950 - FORTE SÃO JOÃO, VITÓRIA - ES, 29017-950

Ambientes dinâmicos no Azure com Terraform

Gustavo Bigardi

Terraform · GitHub Actions · Microsoft Foundry · GitHub Copilot + MCP

PATROCINADORES OFICIAIS

PATROCÍNIO

DIAMANTE



OURO



PRATA



BRONZE



APOIO



Quem sou



Gustavo Bigardi

CTO @ Clarus IT

Microsoft MVP · 7 vezes consecutivas

Esta palestra nasceu de um problema real de release no nosso dia a dia de desenvolvimento.

github.com/gustavobigardi

about.me/gbbigardi

Agenda

01

A dor

Dev compartilhado e a release travada

02

A ideia

Um ambiente efêmero por tarefa

03

Técnicas

Terraform + GitHub Actions na prática

04

Demo ao vivo

Blazor + agente no Foundry, isolado por preview

05

Gestão com Copilot + MCP

Status, custos e limpeza

06

Ganhos e armadilhas

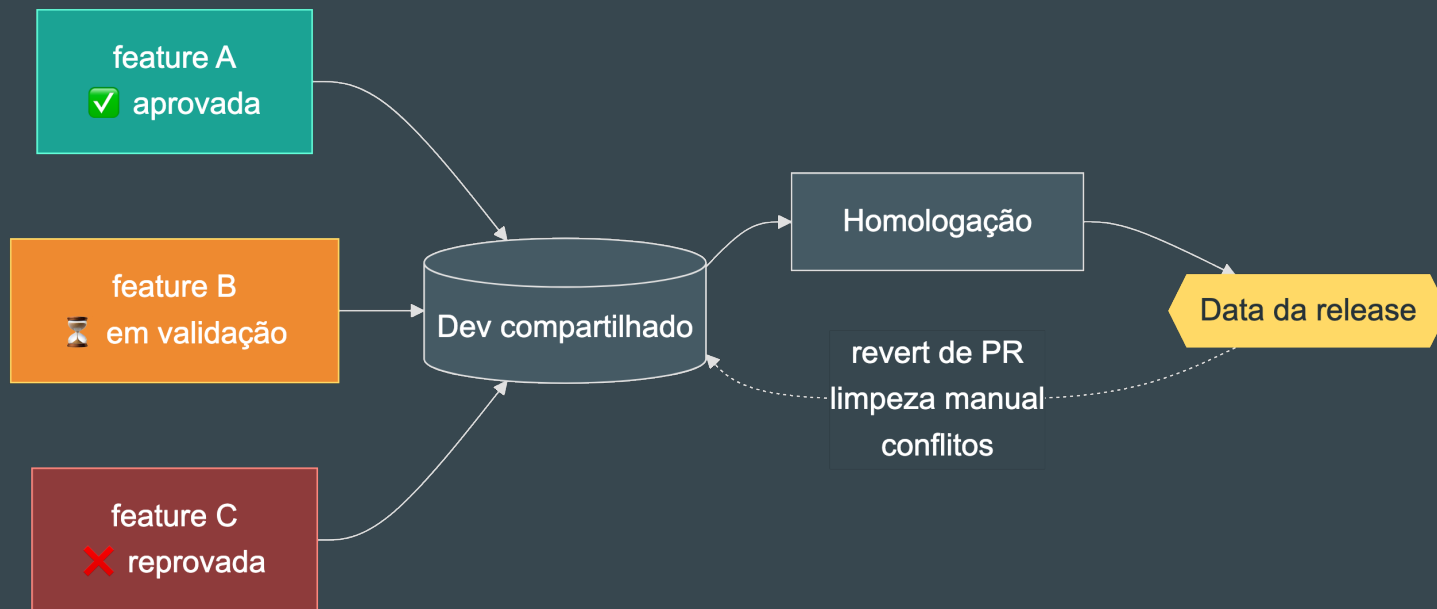
O que aprendemos no caminho

01

A dor

Quando o Dev compartilhado vira gargalo de release

Todo mundo no mesmo Dev



0 dia da release



Data chegou

Aprovadas e não
aprovadas estão juntas no
Dev

Limpeza

Serviço para remover
alterações do ambiente

Revert de PR

Tirar da branch o que
não foi aprovado

Reintroduzir

Revert do revert: conflitos
e retrabalho

“Quem aqui já fez revert de um PR só para conseguir liberar outro?”

O custo que não aparece na fatura

1

Release atrasada

A entrega aprovada espera a limpeza do que não foi aprovado.

2

Retrabalho no Git

Reverts, cherry-picks e conflitos viram parte do processo de release.

3

Ambiente sem credibilidade

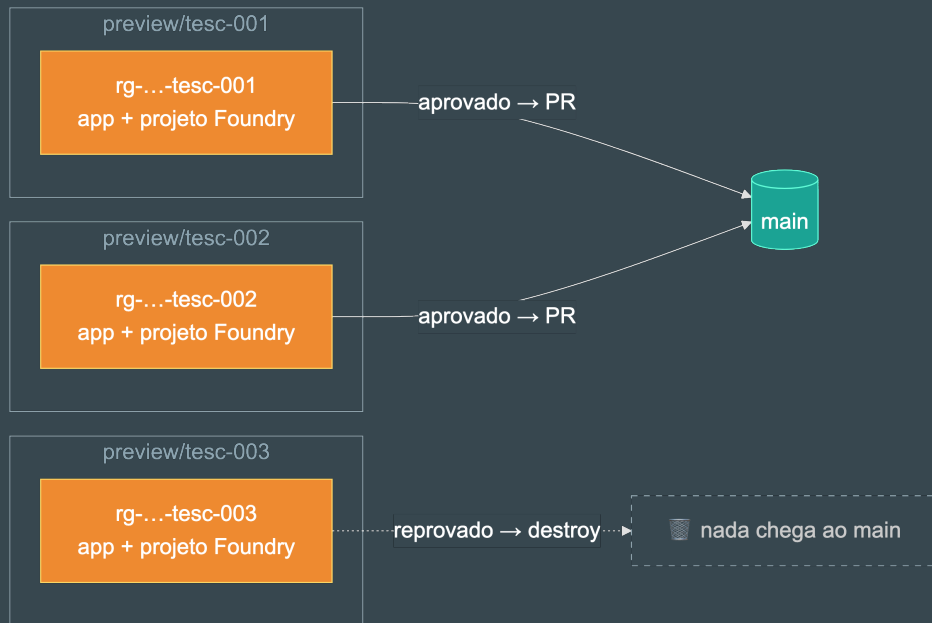
“Funcionou no Dev” deixa de significar alguma coisa para o QA.

02

A ideia

Preview environments: uma cópia efêmera do ambiente por tarefa

Um ambiente por tarefa



Chave

TESC-001

Branch

preview/tesc-001

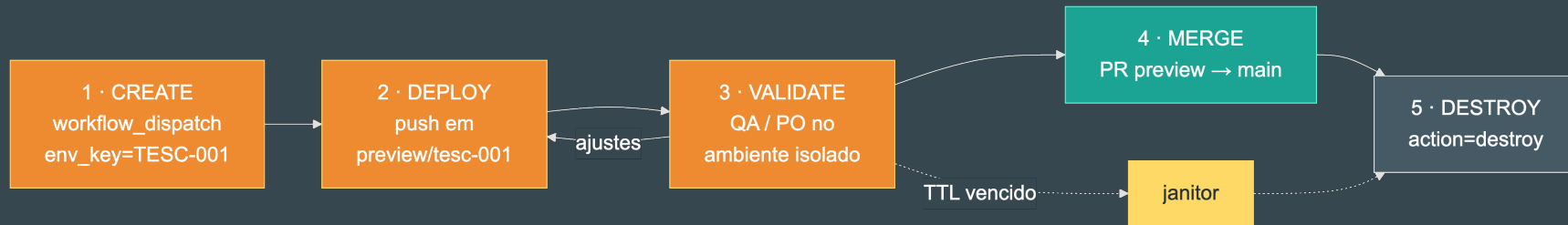
Recursos

*-tesc-001

Vida útil

dias, não meses

Ciclo de vida de um preview



Nasce

`workflow_dispatch` com a chave da tarefa e TTL em dias.

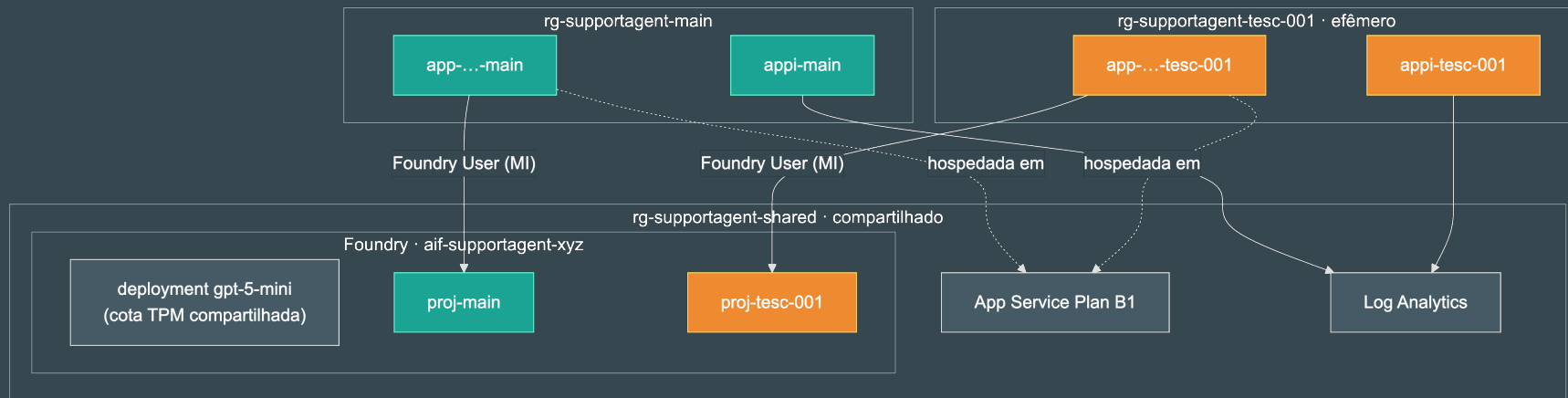
Evolui

Cada push em `preview/<chave>` faz deploy só naquele ambiente.

Morre

Destroy após o merge, ou pelo `janitor` quando o TTL vence.

Arquitetura: compartilhado × isolado



O que compartilhar e o que isolar

Compartilhe

App Service Plan B1

Log Analytics

Foundry resource

Deployment do modelo (cota de TPM)

Regra: caro, lento para criar ou limitado por cota.

Isole

Resource group do ambiente

Web App (managed identity própria)

Application Insights

Projeto Foundry: agente, arquivos, vector store

Regra: carrega estado ou comportamento da feature.

03

Técnicas

Terraform e GitHub Actions que tornam o ambiente dinâmico

State key dinâmica

```
# mesmo código, state diferente
terraform init \
  -backend-config="key=app/${ENV_KEY}.tfstate"

terraform apply \
  -var "env_key=${ENV_KEY}" \
  -var "branch=preview/${ENV_KEY}"
```

main

app/main.tfstate

tesc-001

app/tesc-001.tfstate

tesc-002

app/tesc-002.tfstate

É isso que transforma um Terraform estático em ambiente dinâmico: **a chave é explícita, auditável e vem da branch.**

Um módulo, N ambientes

```
module "environment" {  
  source = "../../modules/app-environment"  
  
  env_key   = var.env_key   # main | tesc-001  
  env_type  = local.env_type # main | preview  
  ttl_days  = local.env_type == "main" ? 0 : var.ttl_days  
  
  # compartilhados descobertos por nome  
  service_plan_id    = data.azure_rm_service_plan.shared.id  
  foundry_account_id = data.azure_rm_cognitive_account.foundry.id  
}
```

Paridade garantida

O main também é criado pelo módulo. Se o preview funciona, o main funciona igual.

Sem acoplar states

Recursos compartilhados via data sources e convenção de nomes.

A branch é o contrato

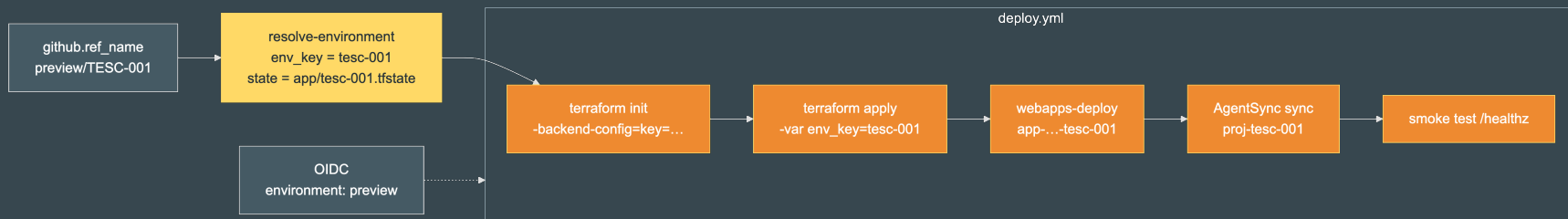
```
# em pull_request, vale a branch de DESTINO
ref="${BASE_REF:-$REF_NAME}"

case "$ref" in
  main)      key="main" ;;
  preview/*) key="${ref#preview/}" ;;
esac
```

Pegadinha

Em pull_request, github.ref_name é “N/merge”.

Use base_ref ou o plan sai no ambiente errado.



Nomes, tags e TTL

```
resource "time_offset" "expiration" {  
  offset_days = var.ttl_days # estável entre applies  
}  
  
tags = {  
  "environment-key" = local.env_key  
  "environment-type" = var.env_type  
  "branch"          = var.branch  
  "expires-at"      = time_offset.expiration.rfc3339  
}
```

Tags em todos os recursos

O Cost Management não herda tags do resource group por padrão.

expires-at estável

time_offset calcula uma vez; mudar o TTL renova.

Janitor diário

Lista vencidos, abre issue ou destrói.

OIDC e menor privilégio

1

Sem segredos

Federated credential por contexto:
environment main, preview e
pull_request.

2

Contributor

A pipeline cria e destrói os
recursos da demo na subscription.

3

RBAC com condição ABAC

Só consegue atribuir o papel
Foundry User às identidades das
Web Apps.

```
repo:gustavobigardi/preview-environments-demo:environment:preview  
@Request[roleAssignments:RoleDefinitionId] ForAnyOfAnyValues:GuidEquals {53ca6127-...} # Foundry User
```

O prompt do agente também é código

```
agent/  
├── agent.json      # nome, modelo  
├── instructions.md # o prompt  
└── knowledge/     # manuais e FAQs
```

Mudar o comportamento do agente segue o mesmo fluxo de qualquer feature: **branch, review, preview, merge.**

PR

O comentário do PR mostra o diff do prompt e o plan do ambiente de destino.

Deploy

AgentSync publica uma nova versão do agente no proj-<env_key>.

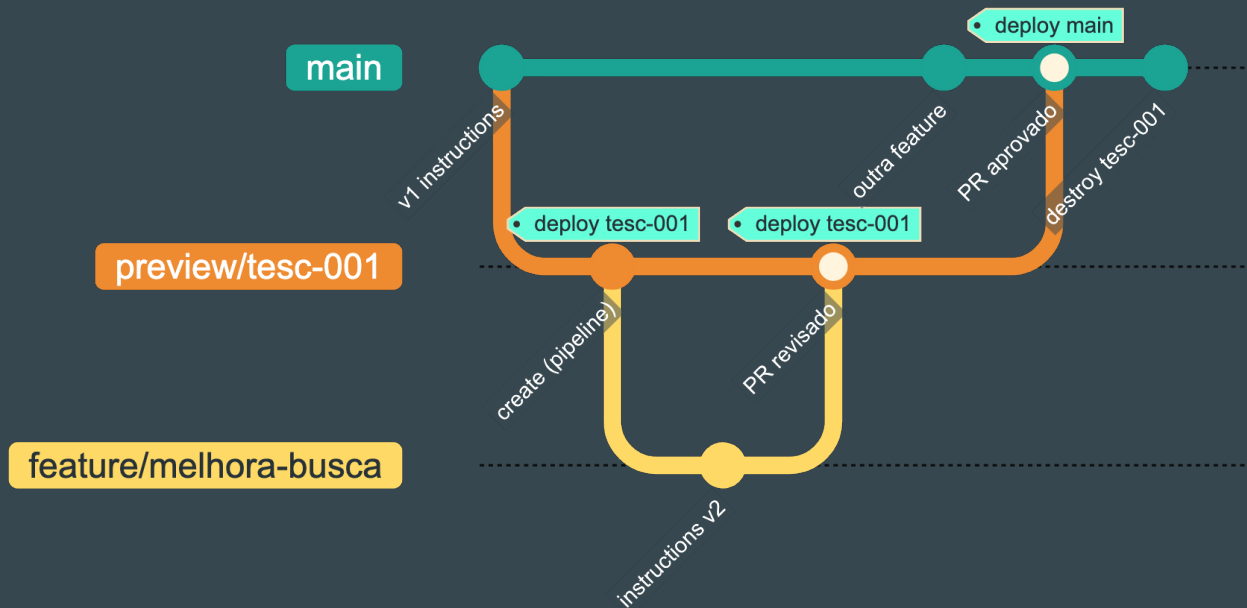
Base

Reindexa o vector store só quando o hash dos arquivos muda.

Destroy

Cleanup de agente, arquivos e vector stores antes do Terraform.

O fluxo completo



04

Demo

Blazor Server + agente no Microsoft Foundry, isolado por preview

O que vamos ver

1

Criar TESC-001

Workflow com a chave da tarefa.

RG, Web App e projeto Foundry surgindo.

2

Mudar o agente só no preview

PR com o prompt v2 para preview/tesc-001.

Comparar main × preview lado a lado.

3

Gerir e destruir

/preview-status e /preview-costs no Copilot.

Merge para main e destroy do preview.

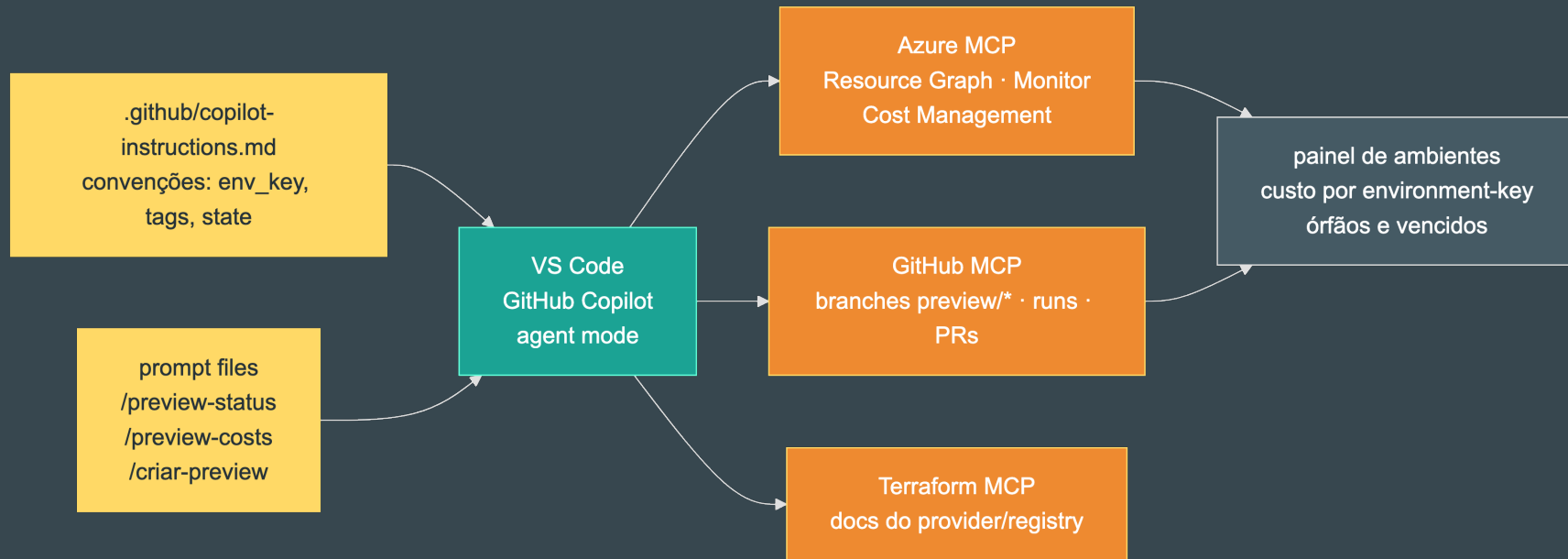
Pergunta-teste: “Como descalcificar minha cafeteira?”

05

Gestão com Copilot + MCP

Status, custos e limpeza dos ambientes em linguagem natural

GitHub Copilot + MCP servers



Prompt files versionados no repositório

`/preview-status`

Previews ativos, último deploy, PRs abertos, expires-at, órfãos e vencidos.

`/preview-costs`

Custo por environment-key, projeção até o TTL e comparação com ambientes dedicados.

`/criar-preview`

Valida a chave, confere conflitos e dispara o workflow após confirmação.

`copilot-instructions.md` ensina as convenções (env_key, nomes, tags, state) para o Copilot responder no contexto certo, e nunca destruir nada fora do workflow.

06

Ganhos e armadilhas

O que melhorou e o que aprendemos no caminho

Ganhos

0

reverts de PR para liberar

O main só recebe o que foi aprovado.

1

ambiente por tarefa

QA valida exatamente a feature, sem interferência.

≈ 0

compute extra por preview

Plano e modelo compartilhados; vida útil em dias; janitor.

Armadilhas que encontramos

Cota de TPM

O deployment do modelo é compartilhado: muitos previews disputam a mesma cota.

Dados do agente

Arquivos, vector stores e versões persistem. Faça cleanup antes do destroy.

Soft delete

Foundry resource tem soft delete; use `purge_soft_delete_on_destroy`.

Limites de nome

Nomes globais curtos: sufixo ≤ 8 caracteres, chave ≤ 20 .

Tokens no compartilhado

O custo do modelo aparece no Foundry, não no RG do preview.

GITHUB_TOKEN

Push feito pela pipeline não dispara workflow: o primeiro deploy é explícito.

Leve para casa

1

1 módulo

para main e previews

2

1 state key

dinâmica por ambiente

3

1 branch

como contrato

Material

Código da demo

[gustavobigardi/terraform-environments-demo](https://github.com/gustavobigardi/terraform-environments-demo)

Terraform + Microsoft Foundry

learn.microsoft.com/azure/foundry

Azure MCP Server

learn.microsoft.com/azure/developer

ENTRE NA NOSSA
COMUNIDADE DO
WHATSAPP!



Obrigado!

Gustavo Bigardi · github.com/gustavobigardi